

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_e1b943dd-dac\agent-a49268fdf04dd9646.jsonl`  
- SHA-256: `f4586dd73909bb93834c835b83af51ce830f4a01b0770425f2042d2dc48913d9`  
- Source modified: `2026-07-06T19:18:32+00:00`  
- Imported at: `2026-07-08T16:00:11+00:00`  
- project: `wf_e1b943dd-dac`  
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`
```

Transcript

1. user (2026-07-06T19:17:45.564Z)

Reviewing GitHub PR #40 of the repo at C:/Users/avidu/Projects/Telegram ? a DRAFT, DOCS-ONLY PR (branch docs/track-f-policy-pipeline).
It adds ONE file: docs/adr/0012-policy-pipeline.md (ADR 0012, status Proposed) ? the Track F design doc for a four-tier "policy pipeline" (Tier 1 metadata gates / Tier 2 topic preferences / Tier 3 content safety / Tier 4 deep inspection), a PolicyStage protocol + StageResult explanation trace, and 7 phased follow-up issues F1-F7.
No code changes. Read the ADR:
git -C C:/Users/avidu/Projects/Telegram show docs/track-f-policy-pipeline:docs/adr/0012-policy-pipeline.md

This is a DESIGN review, not a code review ? there is no gate to run. Judge the ADR as a design contract:
soundness, completeness, internal consistency, accuracy of its claims, and consistency with the codebase and the other ADRs (docs/adr/0001..0011 exist on main; read any you need via 'git -C C:/Users/avidu/Projects/Telegram show main:docs/adr/<file>').
Relevant current code (on main): src/tg_video_filter/models.py (FilterConfig, ContentItem, Source, Policy), src/tg_video_filter/db.py (load_filter_config/save_filter_config shims, sources.watched gate should_ingest_source, delivery_attempts/record_delivery), src/tg_video_filter/telethon_client.py (the live handler: watched gate -> dedup -> FilterEngine.evaluate -> insert -> deliver). Track E (sources.watched opt-in gate) and Track C (delivery_attempts) just merged.

Ground rules:

- READ actual files, cite ADR section / file:line. You MAY run C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe to check claims (e.g. does FilterConfig ignore/drop unknown JSON keys? does ContentItem carry caption text? is content_item.text populated?).
- This is a DRAFT asking for design feedback ? frame findings as design gaps/risks/inaccuracies, not merge-blockers.
 - Severity: high (a claim that is false, or a design gap that would break an F-phase / cause data loss), medium (unspecified interaction, inconsistency, missing dependency), low (wording, invalid example, minor).
- Report only issues in THIS ADR. No praise. Be concrete: what's wrong, why it matters for implementation.

ADVERSARIALLY VERIFY this design finding ? try to REFUTE it. Read the cited ADR section + any referenced ADR/code,
run C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe checks if useful. CONFIRMED if it genuinely holds against THIS ADR (a real gap/inaccuracy/inconsistency).
REFUTED if the ADR actually addresses it / the claim is wrong / it's out of scope for a design doc. PLAUSIBLE if unsettled. Corrected severity (remember: draft design doc, so calibrate ? false claim/data-loss = high, gap = medium).
FINDING (claims): 'topics' example contradicts the prose (array vs object) and is invalid JSON (contains a // comment)
severity medium | section Topic preferences ? Tier 2
summary: Prose says topics are 'stored as a JSON array in policies.config_json under a topics key', but the example stores an OBJECT {include, exclude, mode}. Additionally the example JSON contains a // comment on the 'mode' line, which is not valid JSON and would fail json.loads / model_validate_json if used as a literal config_json value.
why_it_matters: config_json is a real serialized JSON blob (see Policy.config_json, db.py:128-137, load via model_validate_json). A reader who copies the example into config_json gets a parse error. The array-vs-object mismatch also leaves the actual storage shape ambiguous for F2 ('topics config parsing'), which is exactly the contract F2 codes against.
evidence: Prose: 'Topic preferences are stored as a JSON array ... under a topics key'. Example immediately below is an object: {"topics": {"include": [...], "exclude": [...], "mode": "any" // ... }}. Python: json.loads on that example raises 'Expecting , delimiter: line 6 column 20' due to the // comment. FilterConfig.model_validate_json would likewise reject it.
rec: Fix the prose to say 'stored as a JSON object' (or restructure to an actual array if that was intended), and remove the // inline comment from the example ? move the mode explanation into prose or a table, since real config_json cannot carry comments.

2. assistant (2026-07-06T19:17:48.602Z)

I'll verify this finding by reading the actual ADR section and checking the JSON validity claims.

3. user (2026-07-06T19:17:50.436Z)

ADR 0012: Policy pipeline ? explainable, tiered, user-customizable content gates

- **Status**: Proposed
- **Date**: 2026-07-06
- **Decision owner**: project owner
- **Tracking issue**: #31 (Track F of the provider-neutral umbrella, #25)
- **Depends on**: ADR 0011 (Track B ? domain schema, `policies` table)

Context

The system today evaluates every matched video against a single `FilterConfig` (per account, via `policies.config\_json`). The filter is **flat**: all rules run unconditionally, the output is a boolean (`matched`), and there is no way to express topic preferences, security policy, or why a specific video passed or failed.

Track F defines a **policy pipeline** ? an ordered sequence of stages that run before delivery ? so the system can express richer decisions, explain them, and defer expensive work until after cheap rejections.

What we have today (post-Track-E)

Concept	Current state
`FilterConfig` / `Policy`	One JSON blob per account. Flat duration/resolution/size/exclude rules. AND semantics.
`FilterEngine`	Evaluates all rules; returns `FilterResult(passed, reasons)`.
`policies` table	One row per account (is_default=1). `config_json` is the `FilterConfig` blob.

| Security / topic / deep-inspect | Nothing. Codec/keyword/MIME filters and ffprobe were deferred to v2 in the original plan. |

What we want

A **policy** should be a pipeline of ordered **stages**, each with a well-defined cost, inputs, and explainable output. The pipeline runs **after** dedup and **before** delivery. Users should be able to express topic preferences ("I like AI and philosophy videos"), security rules ("reject links from sketchy domains"), and view **why** a specific item was accepted or rejected.

Decision

Stage model ? four ordered tiers

The pipeline is split into four tiers, ordered by cost (cheapest first). Later stages only run if all earlier stages pass:

```
...
????????????????????????????????????????????????????????????
?              Policy Pipeline                                ?
?                                                         ?
? Tier 1 ? Metadata gates  (always runs, zero I/O)           ?
?   duration, resolution, size, codec, mime, GIF/round       ?
?   ? extends today's FilterEngine                           ?
?                                                         ?
? Tier 2 ? Topic preferences (opt-in, metadata-only)         ?
?   source allowlist, keyword match, channel topic tags      ?
?   ? user-expressed interests filter the stream             ?
?                                                         ?
? Tier 3 ? Content safety  (opt-in, metadata + light I/O)    ?
?   URL/domain reputation, file metadata sanity,             ?
?   caption text scanning                                    ?
?   ? cheap checks; no download                              ?
?                                                         ?
? Tier 4 ? Deep inspection (opt-in, heavy I/O)               ?
?   ffprobe, perceptual hash, downloaded content scan        ?
?   ? gated behind explicit user opt-in + budget             ?
????????????????????????????????????????????????????????????
...
```

Stage interface ? `PolicyStage`

Each stage is a pure function (or async function for I/O stages) that takes a `PolicyContext` and returns a `StageResult`:

```
```python
@dataclass
class PolicyContext:
 """Immutable snapshot of what the stage can inspect."""
 content_item: ContentItem # from ADR 0011
 source: Source # the source it came from
 policy: Policy # the owning policy config

@dataclass
class StageResult:
 passed: bool
 reason: str # human-readable explanation
 stage_name: str # e.g. "duration-gate", "url-reputation"
 metadata: dict[str, object] = field(default_factory=dict) # for explainability UI

class PolicyStage(Protocol):
 """A single gate in the pipeline. Stateless; may be async for I/O stages."""
```

```
... async def evaluate(self, ctx: PolicyContext) -> StageResult: ...
```

The pipeline evaluates stages in order and **short-circuits** on the first failure (don't waste I/O on an already-rejected item). The accumulated `StageResult`` list is the **explanation trace** ? every decision is auditable.

## Topic preferences ? Tier 2

Topic preferences are stored as a JSON array in `policies.config_json`` under a `topics`` key:

```
```json
{
  "min_duration_s": 5,
  "topics": {
    "include": ["AI", "Philosophy", "Soccer"],
    "exclude": ["Politics", "Crypto"],
    "mode": "any" // "any" = match any include topic; "all" = match all
  }
}
```
```

Topic matching is **metadata-only** in v1: source title substring match, message caption keyword match, and (future) channel topic tags from Telegram's channel metadata. No ML classification in the design ? that's a separate issue.

## Security stages ? Tier 3

Security stages are **opt-in** and configured per-policy. The design defines three initial stages (implementation deferred to separate issues):

1. **Source allowlist**: reject items from sources not in the `allowlist`` (an explicit list of trusted `provider_source_ids`).
2. **URL/domain reputation**: check links in captions against a configurable blocklist. Metadata-only ? no page fetch.
3. **Duplicate suppression**: enhanced dedup that compares by `content_fingerprint`` across accounts (already handled by `content_items.dedup_key``; this stage adds cross-policy duplicate detection within an account's delivery streams).

Each security stage records its decision in `StageResult.metadata`` for explainability (e.g. `{"blocked_domain": "sketchy-site.com"}``).

## Deep inspection ? Tier 4

Deep inspection (ffprobe, perceptual hash, downloaded content scan) is **gated behind explicit opt-in** because it violates ADR 0003's no-download principle. The opt-in is per-policy (`deep_inspect: true`` in `config_json``) and the stage has a **budget** (max bytes per item, max concurrent downloads) enforced by the policy engine.

This tier is explicitly deferred to a follow-up issue ? the design only **reserves the slot** in the pipeline model so it doesn't require a schema migration when implemented.

## Policy evaluation flow (post-dedup, pre-delivery)

```
```python
async def evaluate_pipeline(
    ctx: PolicyContext,
    stages: list[PolicyStage],
```

```

) -> tuple[bool, list[StageResult]]:
    results: list[StageResult] = []
    for stage in stages:
        result = await stage.evaluate(ctx)
        results.append(result)
        if not result.passed:
            return False, results # short-circuit
    return True, results
...

```

The caller (telethon_client or future Bot API handler) records the results via `delivery\_attempts.last\_error` (for failed items) or a new `policy\_decisions` table (for explainability ? deferred to implementation).

Schema ? minimal changes

****No schema migration**** is required for this design. The `policies` table already holds a `config\_json` JSON blob. The pipeline is a code concept:

- `policies.config\_json` gains optional `topics`, `allowlist`, `security\_checks`, `deep\_inspect` keys ? all backwards-compatible with the existing `FilterConfig` Pydantic model (extra fields are ignored).
- Future `policy\_decisions` table (deferred) would store the explanation trace for debugging: `(content\_item\_id, policy\_id, stage\_name, passed, reason, metadata\_json, evaluated\_at)`.

Implementation phasing (future issues)

Phase	Scope	Dependencies
F1 ? Policy engine prototype	`PolicyStage` protocol, `evaluate_pipeline()`	wire into live-message handler None
F2 ? Topic preferences	`topics` config parsing, source title/caption keyword matching	F1
F3 ? Source allowlist	`allowlist` config, skip non-allowlisted sources	F1
F4 ? URL/domain reputation	Domain blacklist, caption link extraction	F1
F5 ? Deep inspection	ffprobe/perceptual hash, download budget enforcement	F1, ADR 0003 waiver
F6 ? Policy decisions table	Audit trail storage, explainability UI	F1
F7 ? Explainability endpoint	`/why <content_item_id>` command showing the explanation trace	F6

Each phase is a separate, mergeable PR ? none are bundled here. This ADR only defines the ****design contract**** the phases code against.

Consequences

****Positive****

- Unifies topic, security, and quality gates into a single explainable framework ? no ad-hoc if-chains scattered across handlers.
- Cheap gates always run first; expensive enrichment is only reached for items that pass the cheap gates.
- `StageResult` list provides an audit trail (explainability) without additional DB lookups at decision time.
- Backwards-compatible: `policies.config\_json` already exists; adding optional keys requires no migration.
- Each implementation phase is independently shippable behind feature flags.

****Negative****

- Four tiers add conceptual complexity vs. today's flat filter.
- `PolicyStage` protocol requires all stages to conform to the same interface, which may be awkward for deeply different checks (metadata-only vs. I/O-heavy). Mitigated by the `metadata` dict

- catch-all.
- Topic matching is string-based substring matching in Phase 1 ? not semantic. A future ML stage would replace it, but it's good enough for a v1.
- Deep inspection violates ADR 0003 and must be explicitly opt-in with a budget to avoid runaway costs.

Alternatives considered

- ****Do nothing; keep flat FilterConfig.**** Rejected ? the umbrella exists to dissolve the conflation of quality, topic, and security into a single boolean. A pipeline is the natural generalization.
- ****Inline each stage in the handler**** (no protocol). Rejected ? makes it impossible to test stages independently, reorder them, or add new ones without touching the hot path.
- ****Use a rules engine / DSL.**** Rejected ? over-engineered for <100 users. A Python list of callables is simpler, testable, and introspectable.
- ****Bundle all phases into a single PR.**** Rejected ? scope discipline. Each phase is independently shippable; this ADR defines the shared interface they code against.

Open questions for the decision owner

1. ****`StageResult.metadata` shape**** ? dict or typed per-stage? Recommend: dict for Phase 1 (flexible), typed in Phase 2 (once the shape stabilizes).
2. ****Policy decisions table**** ? store every decision, or only failures? Recommend: every decision (explainability), with a TTL-based cleanup for old traces.
3. ****Topic matching**** ? substring only, or regex/glob? Recommend: substring (case-insensitive) for v1; natural-language matching in a follow-up.
4. ****Deep inspection budget**** ? per-item byte limit or per-account bandwidth cap? Recommend: per-item limit (10 MB default) with a per-account monthly cap enforced at the app level.

4. assistant (2026-07-06T19:17:53.703Z)

Now let me verify the two specific claims: (1) the `//` comment makes it invalid JSON, and (2) whether `FilterConfig` ignores extra keys.

5. user (2026-07-06T19:17:57.121Z)

PARSE ERROR: JSONDecodeError Expecting ',' delimiter: line 6 column 20 (char 144)

6. assistant (2026-07-06T19:18:00.817Z)

The `//` comment claim is confirmed. Now let me check the prose wording precisely and the `FilterConfig` behavior.

7. user (2026-07-06T19:18:03.562Z)

OK extra key ignored: `min_duration_s=5 max_duration_s=7200 target_height=1080 max_size_mb=2048 exclude_gif=True exclude_round=True exclude_nosound=False delivery_mode=<DeliveryMode.FORWARD: 'forward'> delivery_target='me'`

8. assistant (2026-07-06T19:18:06.976Z)

Both technical claims verified. Now let me examine the exact prose wording for the "array vs object" contradiction. The ADR text reads:

> "Topic preferences are stored as a JSON array in ``policies.config_json`` under a ``topics`` key"

Then the example shows `"topics": { "include": [...], "exclude": [...], "mode": "any" }` ? which is a JSON **object**, not an array.

Let me confirm the exact wording quoted.

9. user (2026-07-06T19:18:09.344Z)

101:Topic preferences are stored as a JSON array in `policies.config_json``

102:under a `topics`` key:

107: `"topics": {`

174:- `policies.config_json`` gains optional `topics``, `allowlist``,

186:| **F2** ? Topic preferences | `topics`` config parsing, source title/caption keyword matching | F1 |

256:- ADR 0009 ? selective channel scanning (the `topics.allowlist`` concept

10. assistant (2026-07-06T19:18:20.730Z)

All three sub-claims are verified:

1. **Prose says "JSON array"** ? line 101: "Topic preferences are stored as a JSON array in `policies.config_json`` under a `topics`` key". Confirmed verbatim.

2. **Example is an object** ? lines 107-112: `"topics": { "include": [...], "exclude": [...], "mode": "any" }`. This is a JSON object, not an array. Confirmed contradiction.

3. **/// `comment`` makes it invalid JSON** ? `json.loads`` raises `JSONDecodeError: Expecting ',' delimiter: line 6 column 20``. Confirmed. And since `config_json`` is a real serialized blob loaded via `model_validate_json`` (which uses the same JSON parser), a literal copy would fail.

The "why it matters" is also sound: F2's scope is literally `"topics` config parsing"` (line 186), so this example IS the contract F2 codes against, and it's both self-contradictory (array vs object) and unparseable.

Two minor calibration notes that don't refute the finding:

- The `/// comment inside a fenced illustrative block is a common documentation convention; a careful reader recognizes it as annotation. But the finding correctly notes config_json` cannot carry comments and a copy-paste breaks ? a real accuracy gap for a design contract.`
- The array-vs-object mismatch is the more substantive half: it leaves the storage shape genuinely ambiguous, which is a real spec defect.

Severity: this is a documentation-accuracy/spec-ambiguity issue in a draft design doc, not a false design claim or data-loss risk. It would cause confusion/rework at F2 implementation but is trivially fixable and self-evident. Medium is appropriate ? it's a real internal inconsistency in the contract F2 codes against, above pure wording/typo (low).

11. user (2026-07-06T19:18:32.021Z)

Structured output provided successfully